

Simulador de planificación de procesos

**Institución Universitaria
Pascual Bravo**

Sistemas Operativos

Julian Quintero Copete

09/09/2026

Objetivos del Proyecto

- Objetivo principal: Armar un simulador en Python, C++ o Java que actúe como planificador de CPU para manejar las tareas del motor de procesamiento de datos del Sistema Inteligente de Gestión de Tránsito (SIGET).
- **Objetivos específicos:**
 - Mostrar clarito cómo cambian de estado los procesos (Nuevo, Listo, En ejecución, Bloqueado y Terminado).
 - Aplicar dos algoritmos clave: Round Robin (para repartir equitativamente el trabajo) y Prioridades (para atender de una vez las emergencias).
 - Comparar qué tan rápido y eficiente se comporta cada algoritmo frente a varias tareas de tráfico urbano.

Decisiones de Diseño

Para que el programa quedara limpio y cumpliera con lo pedido, se usó Python y se estructuró así:

- La clase Proceso: Guarda la información clave de cada tarea del SIGET, como el nombre, el tiempo que necesita, la prioridad de alerta y los megas de datos. Incluye una función para ir imprimiendo el cambio de estado en pantalla.
- Cola para Round Robin: Usamos `collections.deque` para armar la fila de procesos. Con un *quantum* de 2, las tareas van rotando de forma ordenada. Además, aplicamos una copia profunda (`copy.deepcopy`) para que cada algoritmo arranque con los datos limpios sin interferir con el otro.
- Orden por Prioridad: Para el segundo método, se ordenan las tareas usando funciones sencillas basadas en el nivel de urgencia, asegurando que las emergencias pasen derecho a la CPU.

Funcionamiento de los Algoritmos

A. Round Robin (Quantum = 2)

- Qué hace: Le da un turno de tiempo igual a cada proceso. Si una tarea no alcanza a terminar, se bloquea temporalmente y se vuelve a formar al final de la cola.

- Resultado: Es genial para repartir el trabajo de forma justa y mantener todo fluyendo sin que una sola tarea se apodere de la CPU.

B. Algoritmo por Prioridad

- Qué hace: Revisa qué tarea es más urgente para el SIGET y la ejecuta primero. Una vez arranca, no se detiene hasta terminar por completo.
- Resultado: Asegura que los eventos críticos tengan una respuesta inmediata, aunque las tareas más grandes o menos urgentes tengan que esperar un poco más.

Conclusiones

- Hacer este simulador me ayudó a entender en la práctica cómo los sistemas operativos logran equilibrar el trabajo pesado con la necesidad de reaccionar rápido ante una emergencia en la ciudad.
- La comparación dejó claro que Round Robin es excelente para dar equidad y fluidez al sistema, mientras que el modelo por Prioridades es totalmente obligatorio cuando se trata de seguridad vial y atención rápida.
- Python resultó ser una herramienta súper cómoda para modelar todo mediante objetos y ver los resultados en la terminal de forma clara.